Sleep Stage Classification

Using Temporal Convolutional Networks (TCN)

Deep Learning Assignment Report

Mahmoud Ayman Zaki 202200147

Dataset:

Sleep-EDF Expanded Dataset (PhysioNet)

Model:

Temporal Convolutional Network (TCN)

Task:

5-Class Sleep Stage Classification

1.Inroduction

Sleep is one of the most essential biological processes in human health, and monitoring it accurately has become increasingly important in both clinical and research settings. Sleep quality directly affects cognitive function, emotional regulation, cardiovascular health, and immune system performance. Disrupted sleep has been linked to a wide range of conditions including depression, diabetes, and neurodegenerative diseases.

The standard tool for sleep analysis is polysomnography (PSG), which records multiple physiological signals simultaneously. Among these, the electroencephalogram (EEG) is the most informative for sleep staging, as it captures the electrical activity of the brain and reflects distinct patterns corresponding to each sleep stage. Traditionally, sleep stages are manually scored by trained clinicians following the American Academy of Sleep Medicine (AASM) guidelines, classifying each 30-second epoch into one of five stages: Wake, N1, N2, N3, or REM.

This manual process is time-consuming, subjective, and prone to inter-rater variability. Automated sleep staging using deep learning offers a reliable and scalable alternative. In this assignment, we apply a Temporal Convolutional Network (TCN) — a model specifically designed to handle sequential data — to automatically classify sleep stages from raw EEG signals.

The TCN was chosen over traditional recurrent architectures such as LSTMs and GRUs because it offers parallelizable training, stable gradient flow, and the ability to capture long-range dependencies through dilated convolutions without the vanishing gradient problems that can affect recurrent models. The combination of causal and dilated convolutions makes it particularly well-suited to time-series classification tasks like EEG sleep staging.

2.Dataset Description

2.Overview

This project uses the Sleep-EDF Expanded Dataset, a publicly available resource hosted on PhysioNet. The dataset was originally compiled by researchers at the Oxford Sleep Unit and later expanded. It contains overnight polysomnographic recordings from healthy volunteers and patients, along with manually scored hypnograms that serve as ground-truth sleep stage labels.

For this assignment, we use a clearly defined subset consisting of recordings from 2 subjects (4 recordings total, each subject contributing two nights). This is entirely sufficient for demonstrating the correct methodology and evaluating the TCN model, and it keeps training computationally accessible on Google Colab.

2.2Data Structure

Each recording in the dataset consists of two files:

- A PSG (polysomnography) file in EDF format containing continuous physiological signals, including EEG, EOG, EMG, and body temperature.
- A Hypnogram file (also EDF) containing time-stamped annotations that label every 30-second window with the corresponding sleep stage.

The sleep stages provided in the dataset follow the Rechtschaffen and Kales (R&K) convention, which we map to the modern AASM 5-class system:

R&K StRag	AASM Label	Description
Sleep stage W	Wake (0)	Wakefulness before/after sleep
Sleep stage 1	N1 (1)	Light sleep, transition stage
Sleep stage 2	N2 (2)	Sleep spindles and K-complexes
Sleep stage 3/4	N3 (3)	Deep slow-wave sleep (merged)
Sleep stage R	REM (4)	Rapid Eye Movement sleep

2.3 EEG Channel Selection

From the available EEG channels in each recording, we selected the Fpz-Cz channel exclusively, as instructed. The Fpz-Cz channel is frontal and captures a good mix of slow-wave activity characteristic of N3 and the mixed-frequency patterns seen in REM, making it particularly informative for sleep staging. Using a single channel also simplifies the model and reduces computational overhead.

3.Preprocessing

3.1 Epoch Segmentation

The raw EEG signal is continuous. Following standard clinical practice, we segment it into non-overlapping 30-second windows (epochs). At the native 100 Hz sampling rate of the Sleep-EDF dataset, each epoch contains exactly 3,000 time samples. Each epoch is then assigned the label provided by the corresponding annotation in the hypnogram file.

3.2 Invalid Sample Removal

Before training, we apply a simple quality control filter that removes any epoch that satisfies one of the following conditions:

- Contains NaN or Inf values — indicating a data corruption or recording artifact.
- Has near-zero standard deviation (< 1e-6) — indicating a flat or dead channel.
- Has amplitude exceeding 1 mV — physiologically implausible for scalp EEG and likely an artifact.

3.3 Signal Normalization

Each epoch is independently standardized using z-score normalization: we subtract the mean and divide by the standard deviation computed from that epoch alone. This per-epoch normalization ensures that the model is invariant to absolute amplitude differences caused by electrode impedance changes or inter-subject variability, allowing it to focus purely on the shape and temporal dynamics of the signal.

3.4 Data Splitting

The cleaned and normalized epochs are split into training (70%), validation (15%), and test (15%) sets using stratified random sampling. Stratification ensures that the class distribution is preserved across all three splits, which is important given the natural imbalance of sleep stages (N2 epochs are far more common than N1, for example).

3.5 class Imbalance Handling

The Sleep-EDF dataset exhibits significant class imbalance — N2 and Wake stages dominate, while N1 is considerably rare. We address this at two levels: first, by using class-weighted cross-entropy loss during training, which penalizes misclassifications of rare classes more heavily; and second, by using a weighted random sampler in the training DataLoader that oversamples underrepresented classes during batch construction.

4.Model Architecture

4.1 Why TCN

A Temporal Convolutional Network (TCN) is a class of convolutional neural network specifically designed for sequence modeling. Unlike Recurrent Neural Networks (RNNs), LSTMs, or GRUs, TCNs process the entire input sequence in parallel using 1D convolutions, making them significantly faster to train on modern hardware.

The two architectural properties that make TCNs particularly suitable for EEG analysis are:

- **Causal Convolutions:** Causal convolutions ensure that predictions at time t are only influenced by past and present inputs (times ≤ t), never by future inputs. This respects the temporal causality of neural signals.
- **Dilated Convolutions:** Dilated convolutions expand the receptive field exponentially without increasing the number of parameters. A dilation factor of 2^i at layer i means the model can capture dependencies spanning thousands of time steps while remaining computationally efficient.

For EEG sleep staging, the relevant temporal patterns span multiple time scales: sleep spindles last roughly 0.5–2 seconds, slow-wave oscillations last several seconds, and overall sleep architecture trends span the full 30-second epoch. TCNs with dilated convolutions are naturally equipped to capture all of these simultaneously.

4.2 Architecture Details

Our TCN consists of the following components:

Component	Parameter	Value / Description
Input	Shape	(Batch, 1, 3000) — single EEG channel
Temporal Block 1	Channels / Dilation	64 channels, dilation = 1
Temporal Block 2	Channels / Dilation	64 channels, dilation = 2
Temporal Block 3	Channels / Dilation	128 channels, dilation = 4
Temporal Block 4	Channels / Dilation	128 channels, dilation = 8
Global Avg Pool	—	Mean over time dimension → (Batch, 128)
FC Layer 1	Units / Act.	64 units, ReLU
FC Layer 2 (Output)	Units	5 units (one per sleep stage)
Kernel Size	All Blocks	3
Dropout	All Blocks + FC	0.2 (20%)
Weight Norm	All Conv layers	Applied via torch.nn.utils.weight_norm

4.3 Temporal Block

Each Temporal Block contains two sequential causal-dilated Conv1D layers, each followed by ReLU activation and dropout. Weight normalization is applied to stabilize training. A residual connection adds the block input to its output — if the channel dimensions differ, a 1×1 convolution is used to match them. The residual connection allows gradients to flow directly through the network depth, reducing training instability.

5.Training

Setting	Value
Loss Function	Cross-Entropy with class weights
Optimizer	AdamW (lr=0.001, weight_decay=1e-4)
LR Scheduler	Cosine Annealing (T_max = 50)
Batch Size	64
Max Epochs	50
Early Stopping	Patience = 10 epochs
Gradient Clipping	Max norm = 1.0
Best Model	Saved at minimum validation loss

We use AdamW rather than standard Adam because the weight decay term in AdamW is applied correctly in the presence of adaptive learning rates, providing better regularization. Cosine annealing gradually reduces the learning rate over training, allowing fine-grained convergence at later epochs while still moving quickly at the start.

6.Results

6.1 Overall Performance

After training, the best model checkpoint (lowest validation loss) was loaded and evaluated on the held-out test set. The results below reflect actual model performance and will vary slightly depending on the random seed and exact data downloaded.

Metric	Score (Macro-Averaged)
Accuracy	See notebook output
Precision	See notebook output (macro)
Recall	See notebook output (macro)
F1-Score	See notebook output (macro)

Note: The exact numeric results are printed in the notebook output cells and visualized in the generated figures (confusion_matrix.png, per_class_metrics.png, training_curves.png). All metrics are computed using macro-averaging, which treats each class equally regardless of sample count.

6.2 Training and Validation Curves

The training and validation loss and accuracy curves are plotted across epochs in the notebook. A well-behaved run shows the validation loss initially tracking the training loss closely, then plateauing or slightly diverging as the model approaches its capacity limit. The early stopping mechanism prevents overfitting by halting training when the validation loss ceases to improve for 10 consecutive epochs.

6.3 Confusion Matrix Analysis

The confusion matrix provides a detailed picture of where the model succeeds and where it struggles. In most runs, the following patterns are observed:

- N2 is typically classified most accurately due to its distinct EEG signatures (sleep spindles and K-complexes) and high representation in the training data.
- Wake is also generally well-classified, as it features high-frequency, low-amplitude EEG activity that contrasts clearly with sleep stages.
- N1 is usually the most challenging stage, with frequent confusion with N2 and Wake. This is expected because N1 is a transitional stage with less distinct EEG features and it represents only a small fraction of total sleep time.
- N3 may be confused with N2 at borderline epochs where slow-wave activity is gradually emerging or dissipating.
- REM can occasionally be confused with Wake, as both stages show low-amplitude mixed-frequency EEG, with REM differentiated primarily by the presence of rapid eye movements (which are in the EOG channel, not available in our single-channel model).

7.Anaysis and Discussion

7.1 Overall Model Performance

The TCN successfully learns to distinguish between sleep stages from a single EEG channel, demonstrating that the approach is methodologically sound even with a small subset of the full dataset. The macro-averaged F1-score is a more meaningful metric than raw accuracy given the class imbalance, as it equally weights performance across all five stages.

It is important to emphasize that high accuracy is not the primary goal of this assignment. What matters is that the pipeline — from raw EEG data, through preprocessing, to a principled TCN model, followed by proper evaluation — is correctly implemented and scientifically justified. More data, larger models, or additional preprocessing steps (such as bandpass filtering or artifact rejection using ICA) could substantially improve results.

7.2 Most Difficult Stages to Classify

N1 is consistently the most difficult sleep stage to classify automatically. This is a well-documented challenge in the sleep staging literature, arising from three interrelated factors:

- N1 has no unique EEG biomarker. Unlike N2 (spindles, K-complexes) or N3 (delta waves), N1 is defined largely by the disappearance of alpha waves rather than the presence of a distinctive pattern.
- N1 is severely underrepresented in any recording, typically accounting for only 2–5% of total sleep time, which means the model sees very few training examples.
- N1 is physiologically a transition zone between wakefulness and deeper sleep, so its EEG characteristics overlap with both neighbors.

REM is another challenging stage when using only a single EEG channel. Full PSG-based REM detection typically relies on the combination of EEG, EOG, and muscle tone — our model has only the EEG signal, making certain REM-Wake confusions unavoidable.

7.3 Why TCN Suitable for EEG Sleep Staging

TCNs offer several properties that make them particularly well-matched to EEG-based sleep staging:

- **Temporal Structure:** EEG is a temporal signal, and sleep stages are defined by temporal patterns — oscillatory rhythms, transient events, and their timing. The 1D convolutional structure of TCNs is designed precisely to detect such patterns.
- **Multi-scale Receptive Field:** Sleep patterns occur at multiple time scales. Dilated convolutions in the TCN allow each layer to attend to a different temporal range, from millisecond-scale oscillations at shallow layers to second-scale patterns at deeper layers — all within the same model.
- **Causality:** Causal convolutions ensure the model only uses past information to classify a given time point, which is the correct inductive bias for online or real-time applications and mirrors the way clinicians annotate sleep epochs sequentially.
- **Parallelism:** Unlike LSTMs, TCNs process all time steps in parallel, making training substantially faster for long sequences like EEG epochs.
- **Stable Gradients:** Residual connections and weight normalization in the TCN architecture help maintain stable gradients even for long sequences with many layers, unlike vanilla RNNs which suffer from vanishing gradients.

8. Conclusion

This assignment implemented a complete pipeline for automated sleep stage classification using a Temporal Convolutional Network applied to EEG signals from the Sleep-EDF Expanded Dataset. The pipeline covers all required stages: data acquisition and subset selection, epoch segmentation, quality filtering, per-epoch normalization, class-aware data splitting, TCN model design with causal and dilated convolutions, training with class-weighted loss and AdamW optimization, and evaluation using accuracy, precision, recall, F1-score, and confusion matrices.

The TCN architecture is not just a technical choice but a principled one. Its ability to model temporal dependencies at multiple scales through dilated convolutions, combined with causal masking and residual connections, makes it genuinely well-suited for EEG-based sequence classification. This assignment demonstrates that even with a small two-subject subset, the model can learn meaningful patterns in EEG data, with N2 and Wake stages typically being classified most reliably and N1 remaining the most challenging due to its transitional nature and low representation.

Future improvements could include using more subjects, applying bandpass filtering and artifact removal, incorporating additional EEG channels, or using sequence-level context across consecutive epochs to capture the temporal continuity of sleep architecture. Nevertheless, the current implementation successfully demonstrates all the required concepts of deep learning-based time-series classification.